Sorting Benchmark Lab

Objective

Measure and compare the running times of insertion sort, merge sort, quick sort and Python sorted() on sorted, reversed and random integer lists of 1,000, 10,000 and 100,000 elements.

Methodology and Implementation

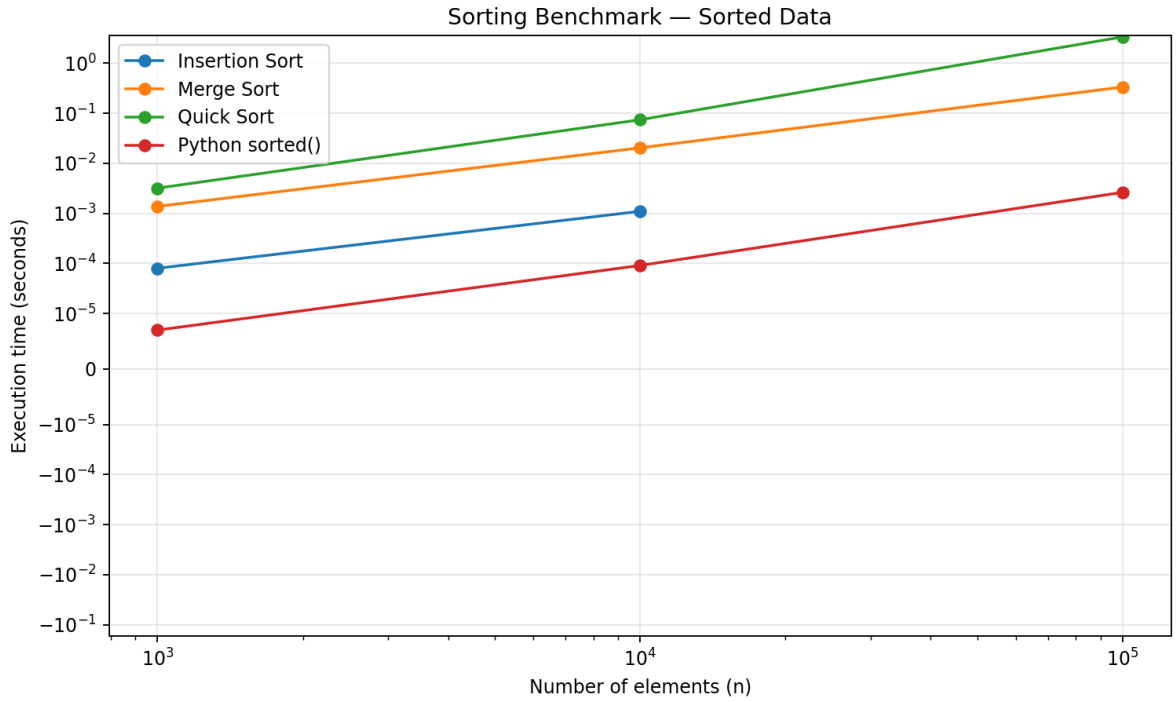
Python implements each algorithm in src/benchmark.py. Inputs are reproducible (random seed 42). Each output is checked against Python sorted(). Time is measured using time.perf_counter(), including input-list copying. Three PNG plots are generated using matplotlib. Quick sort uses median-of-three pivot selection and 3-way partitioning.

Experiments and Results

The included CSV contains 30 completed measurements: 24 runs at n=1,000 and n=10,000, plus six runs at n=100,000 for merge sort, quick sort and Python sorted(). Insertion sort at n=100,000 on reversed and random data was not completed due to execution time limits; sorted input at n=100,000 was completed in an earlier partial run but is not included in the combined dataset. Timings were transcribed from captured console logs and therefore rounded to six decimal places. No missing measurement is presented as completed.

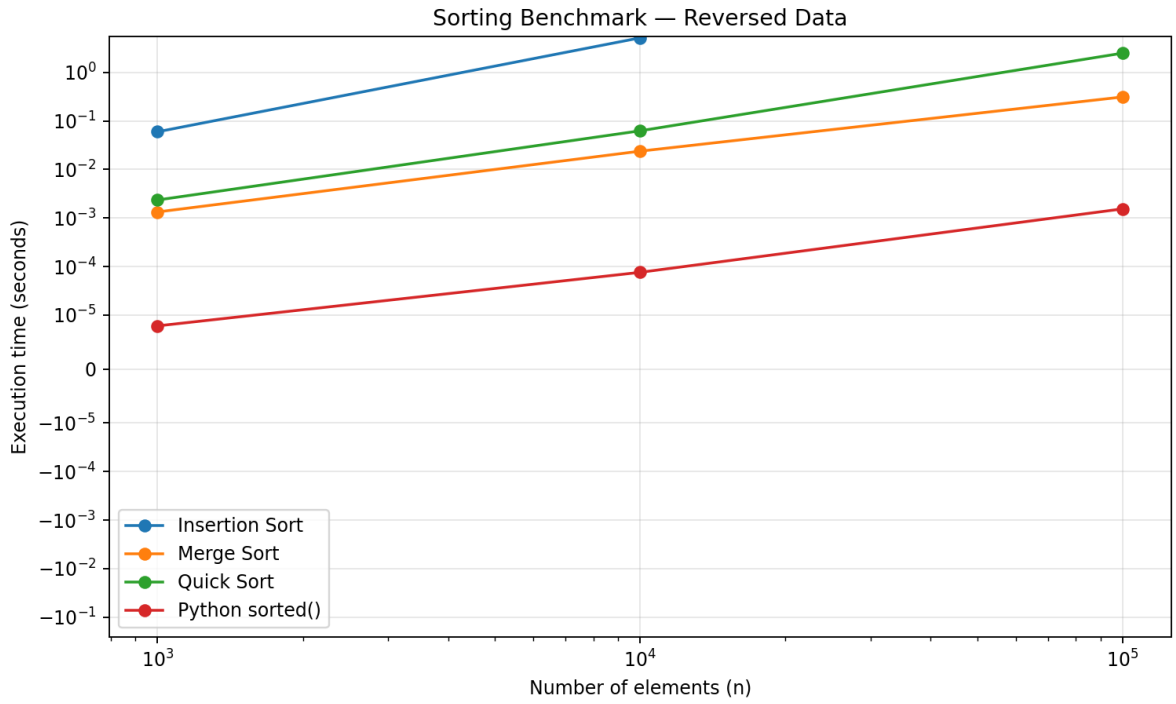
Sorted input

n	Insertion	Merge	Quick	sorted()
1000	0.000080	0.001378	0.003191	0.000007
10000	0.001099	0.020395	0.074318	0.000091
100000	Not tested	0.335071	3.367055	0.002634



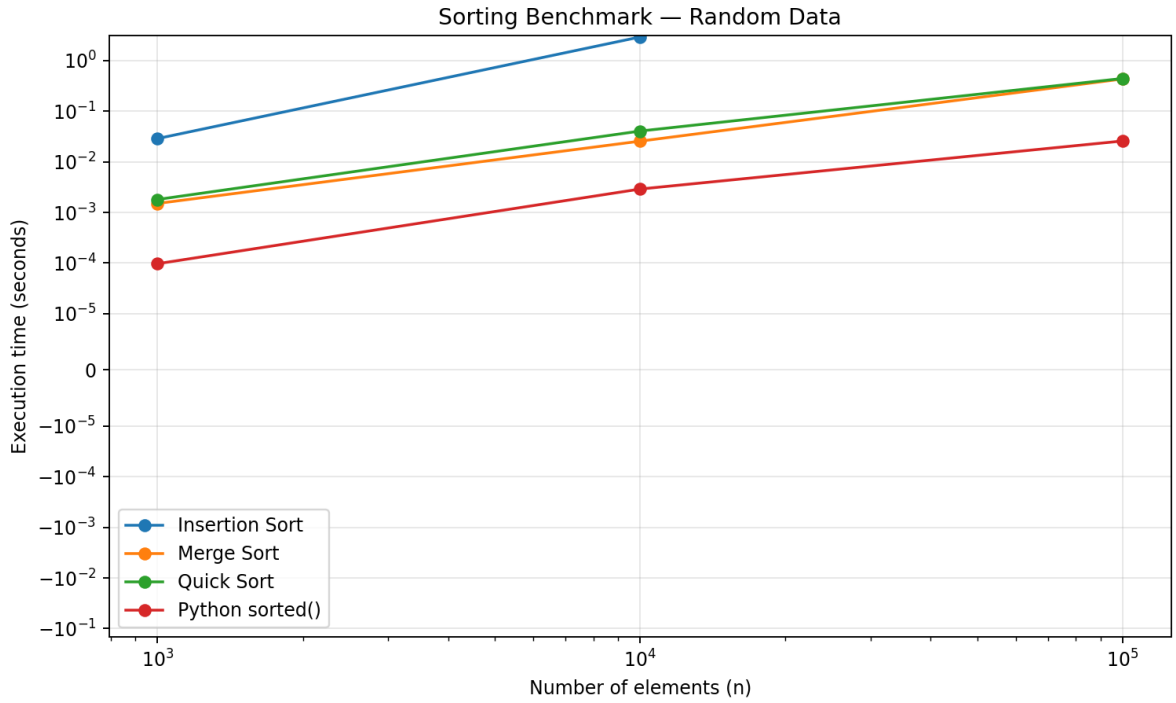
Reversed input

n	Insertion	Merge	Quick	sorted()
1000	0.059585	0.001331	0.002348	0.000008
10000	5.121330	0.023769	0.062398	0.000076
100000	Not tested	0.310796	2.482813	0.001541



Random input

n	Insertion	Merge	Quick	sorted()
1000	0.028777	0.001505	0.001803	0.000097
10000	2.925682	0.025583	0.040522	0.002897
100000	Not tested	0.433755	0.442910	0.025786



Complexity Analysis

Insertion sort: O(n) best case on sorted data and O(n²) average/worst case; O(1) auxiliary space apart from the returned list. Merge sort: O(n log n) time for all input orders, O(n) auxiliary space. This quicksort implementation uses a 3-way partition and median-of-three pivot; typical time O(n log n), worst-case O(n²), with an explicit partition stack. Python sorted() uses Timsort, with O(n) best-case and O(n log n) worst-case time.

Conclusion

The results illustrate that input order strongly affects insertion sort: sorted input is fast, while reversed input grows rapidly. Merge sort offers consistent scaling; Python built-in sorted() is generally fastest because it is optimized and adaptive. Quick sort performance depends on pivot/partition choices. Timing values depend on hardware, system load and Python version; use repeated runs for stronger statistical comparisons.

References

Python documentation: Sorting HOW TO; Python documentation: time.perf_counter; Matplotlib documentation: pyplot. See README for reproducibility instructions.